

Customer Solutions Engineer Interview Assessment

Submitted by Wes Jones

Submission links

GitHub repository: <https://github.com/web3wes/DBT-labs-takehome-final>

Successful dbt Cloud run:

<https://ze846.us1.dbt.com/deploy/70506183140950/projects/70506183141978/runs/70506183440619>

Run status: Success | **Run #**70506183440619 | **Commit #**38e24157 | **Commands:** dbt seed --select tag:prod, dbt build --select tag:prod

This document contains the written responses for Part 1 and Part 2. The repository linked above contains the dbt project, seed data, models, tests, and setup notes.

Part 1: Customer Billing Scenario

1. Initial steps and why

I would treat this as both a support issue and an account-context issue.

The customer is frustrated by unexpected overage charges, and there is also an active Enterprise opportunity in Salesforce. The response needs to be technically useful without turning into a premature sales motion.

My initial steps would be:

- **Confirm the billing facts.** I would verify the current plan, contracted run allowance, monthly usage totals, overage calculations, and when the customer first began exceeding the 15,000 monthly run allowance.
- **Inspect run history.** I would group runs by job, environment, trigger type, schedule, and status to find the largest contributors.
- **Look for avoidable run volume.** I would check for high-frequency schedules, duplicate jobs, repeated failures/retries, excessive CI runs, API-triggered runs, and production jobs that could be scoped differently.
- **Review freshness requirements.** I would compare each high-volume job against its business SLA. A job running every 15 minutes may be necessary, or it may simply be historical habit.
- **Coordinate internally.** Because there is an active Enterprise opportunity, I would align with the Account Executive or CSM before responding in depth.
- **Propose two tracks.** I would separate immediate Team-plan optimization from a longer-term plan review if the customer's legitimate production needs consistently exceed the Team-plan allowance.

This approach helps avoid optimizing around incomplete data while also protecting customer trust. I would not make pricing promises, offer credits, or position Enterprise as the only solution before completing the usage review.

2. Draft response to the customer

Hello,

Thanks for reaching out, and I understand why this is frustrating. Overage charges are especially painful when they feel unpredictable. We can help you review where the run volume is coming from and identify ways to reduce unnecessary runs before you make any decision about moving to Enterprise.

The first thing I would like to do is review your recent dbt Cloud run history and group usage by job, environment, trigger type, and schedule. That will help us distinguish between runs that are essential for production freshness and runs that may be avoidable, duplicated, or scheduled more frequently than needed.

Specifically, I would look for opportunities to:

- align high-frequency schedules with actual freshness requirements
- consolidate duplicate or overlapping jobs
- avoid scheduled runs when no new source data is available
- make CI and development runs more targeted with selectors, state comparison, and deferral
- reduce run volume caused by failures and repeated retries
- separate models by SLA so critical work runs frequently and slower-changing models run less often

My recommended next step is a usage review session where we look at the last one to three billing periods and identify the top contributors to run volume. After that, I can provide a concrete optimization plan, such as:

- jobs to disable or consolidate
- schedules to reduce
- models to separate by freshness tier
- CI commands to make more selective
- jobs that should become event-driven rather than interval-based
- monitoring thresholds so future overages are visible earlier

If the review shows that your current run volume is primarily driven by legitimate production needs, then it may make sense to revisit plan structure with your account team. But I would recommend first completing the usage review so that any plan discussion is based on actual workload requirements rather than surprise charges.

Best,

Wes Jones

3. Risks and considerations

- **Data freshness risk:** Reducing run volume can make dashboards stale if schedule changes are not tied to business SLAs.
- **Job consolidation risk:** Combining jobs may reduce invocations, but it can increase blast radius if a larger job fails.
- **CI coverage risk:** More selective CI is useful, but it should not remove tests that protect shared models.
- **Event-driven orchestration risk:** API or webhook triggers can reduce unnecessary scheduled runs, but only if upstream signals are reliable.
- **Commercial risk:** Because an Enterprise opportunity is active, Support should avoid pricing commitments or implying that Enterprise is required before the technical review is complete.

The guiding principle is to reduce waste without weakening production reliability or customer trust.

4. Alternative solutions considered

Alternative	Why I would not prefer it
Immediately recommend Enterprise	The customer specifically asked to explore current-plan options first. Leading with an upgrade could feel dismissive.
Reduce all schedules by a fixed percentage	This ignores business criticality. Some jobs may need frequent execution, while others may be safe to run daily.
Focus only on model performance	Faster jobs may reduce warehouse cost, but the billing issue is monthly run volume.
Disable CI or development jobs aggressively	This can create quality and deployment risk. A better path is selective CI using state-based selection and deferral.

Part 2: dbt Cloud Setup Process

For this setup, I used:

- **dbt Cloud:** developer account
- **Version control:** GitHub
- **Warehouse:** hosted Neon PostgreSQL database
- **Sample data:** seeded ecommerce data for users, orders, order items, and products

I initially explored BigQuery because it provides public datasets. However, the available GCP project enforced a policy that prevented service-account key creation. Since the Starter/Team-style setup path for BigQuery expects a service JSON key for deployment environments, PostgreSQL was the more reliable path for completing the end-to-end assessment workflow.

Setup summary

I connected dbt Cloud to the GitHub repository with a deploy key, created a dbt project, and connected the project to PostgreSQL. I used seeded ecommerce data because it still contains realistic entities such as users, orders, order items, and products, while keeping the setup reproducible in a free hosted database.

The production deployment environment used a dedicated PostgreSQL schema for dbt-built objects, separate from the development schema. I created a deploy job in the production environment and scheduled it to run every 12 hours, which was the least frequent interval available in the dbt Cloud interval scheduler during setup.

The production job was also validated with a successful manual run: Run #70506183440619 completed successfully on commit #38e24157 after running `dbt seed --select tag:prod` followed by `dbt build --select tag:prod`.

1. Objective and transformation logic

The objective of the dbt transformation job was to turn raw ecommerce sample data into a small analytics-ready mart that could answer questions like:

- How much revenue did we generate by day?
- How many orders were placed?
- What was the average order value?
- Which product categories contributed most to revenue?
- How do customer cohorts or regions perform over time?

I structured the project in three layers:

Layer	Purpose	Models
Staging	Clean source-aligned tables, standardize names and types, keep lineage easy to follow	stg_orders, stg_order_items, stg_users, stg_products
Intermediate	Centralize reusable join logic between orders, order items, and products	int_order_items_enriched
Marts	Produce business-facing reporting models	fct_daily_sales, dim_customers

The marts answer practical analytics questions:

- `fct_daily_sales` aggregates completed order-item revenue by order date, product category, and department.

- `dim_customers` provides customer-level attributes and first-order information.

I added basic tests for:

- unique and non-null keys
- relationships between fact and dimension models
- accepted values for status fields

The production job used commands similar to:

```
dbt seed --select tag:prod
dbt build --select tag:prod
```

I included `dbt seed` because the sample dataset is versioned with the project. I prefer `dbt build` after seeding because it runs models, tests, snapshots, and seeds in DAG order where applicable. For a small production mart, that gives a compact but complete deployment command.

2. Challenges and how I addressed them

Challenge	How I addressed it
BigQuery service-account key creation was blocked by the GCP organization policy <code>constraints/iam.disableServiceAccountKeyCreation</code> .	I switched to PostgreSQL, which dbt Cloud can connect to with host, database, username, password, and schema credentials.
GitHub was not linked inside the dbt Cloud browser session.	I used the Git clone workflow and added dbt Cloud's generated deploy key to the GitHub repository.
Development and production credentials needed to stay separate.	I configured a development schema for development work and a production deployment schema for scheduled job outputs.
The default Fusion release track did not support the PostgreSQL adapter.	I changed the deployment environment to the Compatible release track aligned to dbt Core before creating the production job.

The resources I used were the dbt Developer Hub setup documentation, the GitHub connection documentation, the BigQuery connection documentation while investigating the original path, the PostgreSQL connection documentation, and the deployment jobs documentation.

3. Improvements to the dbt Cloud setup process

- **First-run checklist:** Provide a clearer guided sequence: create project, connect Git, connect warehouse, test credentials, create development schema, create deployment environment, run a sample model, and schedule a production job.
- **Warehouse permissions:** Surface warehouse-specific permission guidance earlier. For BigQuery, the UI could warn users when organization policy disables service-account key creation and point them toward supported alternatives.
- **Environment clarity:** Explain the difference between development credentials, deployment credentials, development environments, and production deployment environments more explicitly.
- **Error messages:** Make connection-test failures more prescriptive by naming the missing resource, missing action, and likely permission needed.
- **Scheduling guidance:** Show the next several scheduled run times in both UTC and the user's local time, and explain available schedule granularity.
- **Guided sample projects:** Offer an optional sample project path per warehouse. For PostgreSQL, that could generate seed files, starter staging models, basic tests, and a first production job.

References consulted

- dbt Developer Hub: [BigQuery quickstart](#)
- dbt Developer Hub: [Connect to GitHub](#)
- dbt Developer Hub: [Connect BigQuery](#)
- dbt Developer Hub: [Connect PostgreSQL](#)
- dbt Developer Hub: [Deployment environments](#)
- dbt Developer Hub: [Deploy jobs](#)
- dbt Developer Hub: [Job scheduler](#)
- dbt Developer Hub: [Defer](#)